

GAME1402 – Assignment 3

Complete Game Project

Technical Document Written by: Kyle MacLachlan

AI Systems

1) NPCs

NPCs in this game include the Ranger, and the Rogue. Both characters are set to wander in a set radius around their spawn point. Their dictated direction is set to randomize as they wander. When the player stands in front of the player in a “Trigger Box Collider” the player can interact with the characters and get one of three random messages displayed on a Toast

2) Slime

The Slime is an NPC that hops about to set patrol points. The slime shifts animation based on whether he is patrolling, chasing, and attacking. The animation does function properly.

Troubleshooting: When the slime interacts with the player, the player should be taking 1 hit of damage. The player does not take damage despite the code written using references from previous game codes. Additionally, the slime should be defeated or killed when the player shoots the slime with an arrow. This mechanic does not function as well, despite the code also being built. The structure uses an IDamageable function that SHOULD do damage to both the player and the slime.

Refinement and Iteration

Using the feedback from Assignment 2 I addressed the following issues:

- Toast.cs and LevelClear.cs contain nearly identical singleton implementations and similar show/hide methods.

Refinement: created a singleton script that operates the show and hide mechanics. It calls on the Toast.cs script and Levelclear.cs script to implement.

- Player Controller is quite large and handles multiple responsibilities. Consider extracting methods like CalculateMovementExplore() and CalculateMovementAim().

Refinement: created a MovementHandler Script that takes all the Calculate Movement Codes out of Player Movement. This helped reduce the script size.

- Crosshair.cs has an unused method State() that is never called.

Refinement: removed State().

- AudioManager contains a typo in the method.

Refinement: corrected spelling.

- PlayerInteractor.cs contains commented out code and redundant assignment logic. The _tempInteractable variable is created and immediately assigned to _interactable.

Refinement: removed and recoded PlayerInteractor.cs to remove the _tempInteractable conditions.

- Several Debug.Log statements remain in code.

Refinement: Removed Debug.Logs

- Consider creating reusable methods for common patterns: sound effect playback with null checks is repeated across multiple PlayXXX() methods.

Refinement: Created a helper in code,

- More classes should utilize the IInteractable interface pattern.

Refinement: The moving platforms code were removed since the Moving Platforms do not exist in file. IInteractables were reconstructed to follow an inheritance structure along with NPC.cs

Event Architecture

The project uses event-based communication to decouple gameplay systems. For example, the Shooter.cs script raises events such as AttackEvent, which are subscribed to the PlayerAnimator. This allows animation logic to react to gameplay actions without tightly coupling the systems together.

The game uses a PlayerState enum to manage player behavior across different gameplay modes such as exploration and aiming. State changes are broadcast using events, allowing systems like UI and animation to respond consistently and maintain synchronization.

Manager classes such as GameManager and AudioManager are used to coordinate gameplay systems. The GameManager handles game flow events such as pausing, player

death, and level transitions, while the AudioManager centralizes sound playback and volume control.

The project demonstrates a decoupled architecture by separating responsibilities across systems. Interfaces such as IDamageable allow different objects to respond to damage without requiring specific references. Event-driven communication further reduces dependencies resulting in modular and maintainable code.

The project uses an event-driven architecture to enable communication between systems without tight coupling. Gameplay states are managed through a PlayerState system, ensuring consistent behavior across movement, animation, and UI. Manager classes such as GameManager and AudioManager coordinate global systems, while interfaces like IDamageable promote modular and reusable interactions. This results in a clean, scalable, and maintainable codebase aligned with professional development practices.

Complete Game Loop

There is a win-loss condition in the game. If the player reaches the end of the level, they win. If the player goes into the water. The player dies.

Troubleshooting: If the player took enough damage from the slime, the player would also die. This functionality does not function, despite the code looking and feeling operational.

Code Quality

I've worked to ensure C# naming conventions in the code are accurate and professionally designed. Privates are `_variableName`, protected and `[SerializeField]` privates use the public `variableName` design.

Git repository uses meaningful naming set up to ensure accurate updating. Branch was developed to separate Assignment 2 from Assignment 3.

Scripts are sorted and organized based on purpose and use:

Enemy folder, Interactable folder, Managers folder, NPCs folder, Player folder, and UI_UX folder.

Technical Document Follow-up

I've covered the complete technical documentation for systems and architecture, outlined in the elements above.

I've played my game for developer feedback. I've had classmate's playtest the game to examine "purposeful breaking and glitching". Things like being able to jump on top of trees, and other actions. I've had my brother and family playtest as "No hold back" feedback.

Despite what Mike Grylls has said about family holding back on feedback to prevent hurt feelings. My family knows that I want to be an amazing game developer. They will try to do things that even I, or our class won't think of doing, to see how the game plays, and will provide their feedback on "This plays great, could you do this", or "this has an issue here, maybe address this issue". This was how we discovered that button mashing the attack button will cause rapid-fire of arrows, even though the game doesn't fire an arrow until a certain point by Coroutines. This bug was also addressed and corrected.

Code was reviewed in person on April 15th with Indy, addressing review of NPC.cs (an abstract code) and other new elements.

AI was used to understand the feedback written by Indy from assignment 2.

AI may be used to understand and approach code in pseudocode format, since that may be a better approach to addressing complex systems, and get an understanding of how Code works outside of class instructions.